

Monster Hunter Rise 护石评测

基于标签传播算法的装备分级系统

小组成员：唐晨宇, 沈豪, 陈高廷, 沈宇涵

指导教师：雷杰

(Dated: December 31, 2021)

摘要: Monster Hunter Rise 这个游戏中有一种刷装备的机制，护石的可用性决定了它的价值。在本项目中，我们利用机器学习来进行护石的分析，利用标签传播算法进行未标记数据的标记，从而得到护石的价值分类。在经过数据的一定分析后，我们按对数据的处理、特征的提取、模型的构建与实际的应用来进行本课程设计的实施。

关键词: 半监督，机器学习，数据分析，特征降维（PCA，LDA），归一化，标签传播算法

目录

1 项目概要	2
1.1 项目背景	2
1.2 项目介绍	2
2 数据模块	2
2.1 数据收集	2
2.1.1 护石数据	2
2.1.2 技能数据	3
2.2 数据处理	4
2.3 数据分析	4
2.3.1 各个 value 的护石的数量及占比	4
2.3.2 每种等级的数量	5
2.3.3 value 与技能的关系	5
2.3.4 护石等级与 value 的关系	6
2.4 特征提取	7
2.4.1 降维	7
2.4.2 降维	7
3 机器学习模型	8
3.1 半监督学习	8

3.2	标签传播算法	9
3.3	sklearn 模块应用	9
3.4	应用模型生成	10
4	应用场景	10
4.1	GUI	10
4.2	护石判断	11
5	特色	14
6	任务分工	14
7	分析总结	14

1 项目概要

1.1 项目背景

Monster Hunter Rise 这个游戏中有一种刷装备（护石）的机制，护石上会带有一些技能和孔（技能插槽），玩家会根据技能以及插槽的好坏选择是否保留该护石或者作为材料融合成新护石。而有些新手玩家不知道护石可不可用，可能会存在一定将不错的护石融合掉的情况。面对这种情况，我们利用机器学习来进行护石的分析，用来进行辅助判断。得到一个护石的基本评分。

1.2 项目介绍

在本项目中，我们首先在网上收集了 Monster Hunter Rise 中的一些护石数据，通过对数据表的分析与日常经验，我们标记了一部分数据，我们对数据进行了一定的特征分析与特征提取，并且通过 Label Spreading 这一半监督模型进行未标签数据的标记，并对处理后的数据在进行一次数据的分析，最后通过各种全监督的模型进行测试并保存模型为后续应用作准备。

在应用方面，我们绘制了一个简单的图形用户界面，利用所训练的模型，将输入的数据通过模型进行预测，并输出护石的预测等级。

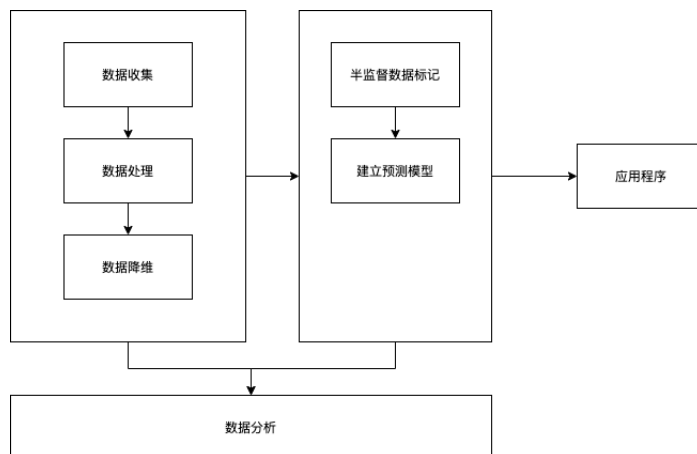


Figure 1: 项目架构

2 数据模块

2.1 数据收集

我们的数据是从网上收集而来的 MHR 的玩家在游玩过程中实际刷取出来的护石数据与游戏中护石可能带有的所有技能及其最大等级。

2.1.1 护石数据

我们收集到的数据集中，每一个护石一共有 5 个特征值，包括等级，护石自带的技能与每个技能对应的技能等级，孔位。我们通过这些数据手动将每一个护石进行评级（1-5 分），并将评级置于 value 列

中。应为数据量较为庞大，我们选择手动输入了 500 条，然后通过标签传播的方式来半监督学习，从而获取到更多的数据，在得到之后我们也手动调整了其中误差较大，不合理的评分，最终获得了 1500 条数据。

```
护石:
  等级  一技能  Lv  二技能  Lv1  孔  value
0    7  火属性攻击强化  1  不屈  1.0  21    2
1    6  昏厥耐性  2  击晕术  1.0  11    3
2    5  植生学  1  毒属性强化  2.0  11    1
3    6  风压耐性  1  击晕术  1.0  11    1
4    7  逆袭  2  利刃  1.0  1    4
基本信息:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1499 entries, 0 to 1498
Data columns (total 7 columns):
等级      1499 non-null int64
一技能    1499 non-null object
Lv        1499 non-null int64
二技能    1341 non-null object
Lv1       1341 non-null float64
孔        1499 non-null int64
value     1499 non-null int64
dtypes: float64(1), int64(4), object(2)
memory usage: 82.1+ KB
None
```

Figure 2: 护石数据集

2.1.2 技能数据

技能数据集中一共有 113 条数据，即 112 个技能与一个“无”，这个数据集用于处理护石数据集中技能文本特征，将文本转为数字，也可用于玩家输入时选择。

```
技能:
  LookUpName  MaxLevel
0      攻击           7
1    挑战者           5
2      无伤           3
3      怨恨           5
4    死里逃生           3
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 113 entries, 0 to 112
Data columns (total 2 columns):
LookUpName    113 non-null object
MaxLevel      113 non-null int64
dtypes: int64(1), object(1)
memory usage: 1.8+ KB
None
```

Figure 3: 技能数据集

2.2 数据处理

数据的初步处理主要有两部分。一是：将空值用指定内容填充（将二技能和其等级的空值分别填充）；二是：对技能根据技能数据集用对应的数字替换。

```
# 处理缺失值
stone.fillna({"二技能": "无", "Lv.1": 0}, inplace=True)

# 处理技能, 将技能名称用数字替换
skillArray = skill["LookUpName"].values
stone['一技能'].replace(skillArray, list(range(1, 113))
stone['二技能'].replace(skillArray, list(range(1, 113))
stone.head(10)
```

	等级	一技能	Lv	二技能	Lv1	孔	value
0	7	13	1	92	1.0	21	2
1	6	76	2	41	1.0	11	3
2	5	79	1	18	2.0	11	1
3	6	62	1	41	1.0	11	1
4	7	106	2	23	1.0	1	4
5	4	40	3	16	1.0	0	4
6	6	67	2	69	1.0	21	1
7	5	70	1	13	2.0	11	2
8	5	18	2	53	1.0	1	1
9	7	76	1	62	1.0	21	2

Figure 4: 数据初步处理

2.3 数据分析

2.3.1 各个 value 的护石的数量及占比

护石数据集中, value 为 1 和 2 的占了大多数, 即不好的护石, 在数据中的占比最大, value 为 4 和 5 的仅占了极小一部分, 最好用的护石的数量及其稀少。

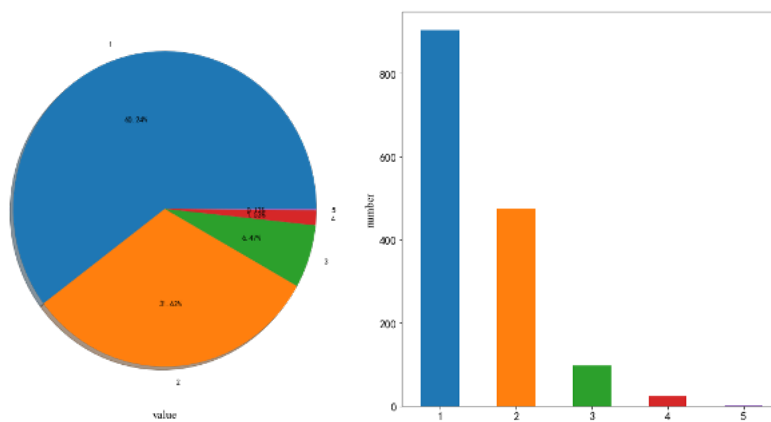


Figure 5: 各个 value 的护石的数量及占比

2.3.2 每种等级的数量

其中等级为 5 和 7 的护石数量最多

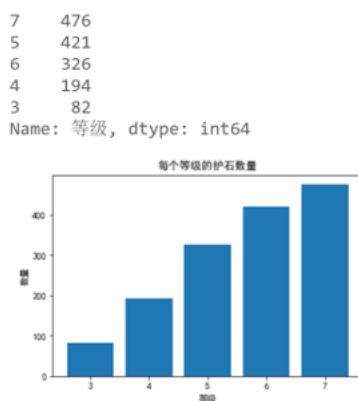


Figure 6: 每种等级的数量

2.3.3 value 与技能的关系

所有技能对 value 的影响都不同, value 值均值越高, 说明该技能越有用, 图中分布显示有的一技能 (如 2,11,13 等) 能有效提升护石的 value。从图中可知, 技能所在一技能或二技能的位置不同时, 对护石 value 的影响是不一致的, 二技能中反而是 30, 53 号等技能能给护石带来较高的 value 的数值。value 与孔位的关系。

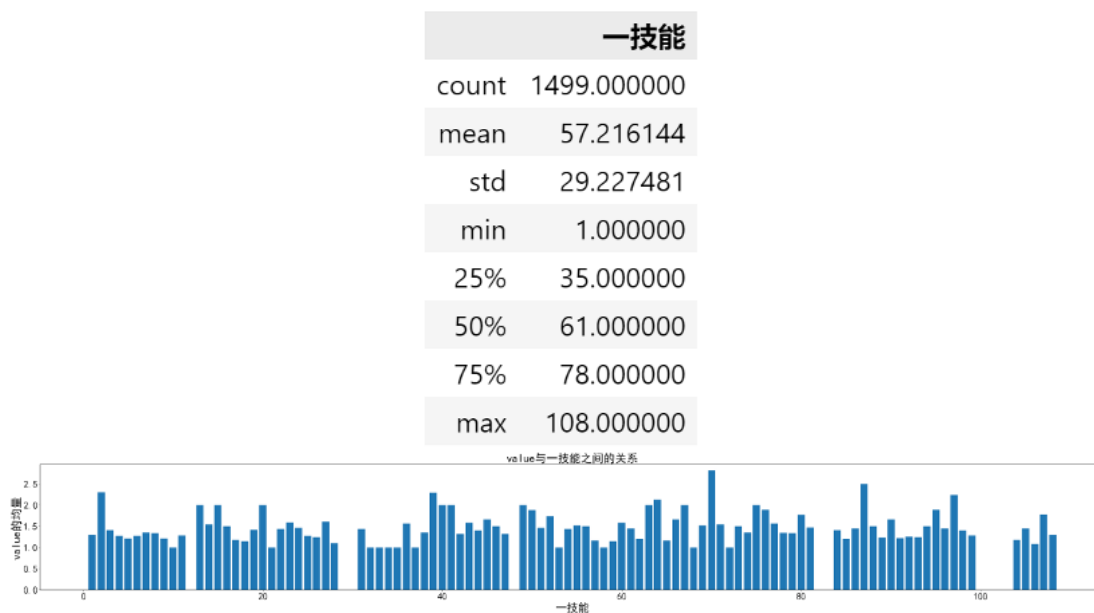


Figure 7: 技能一

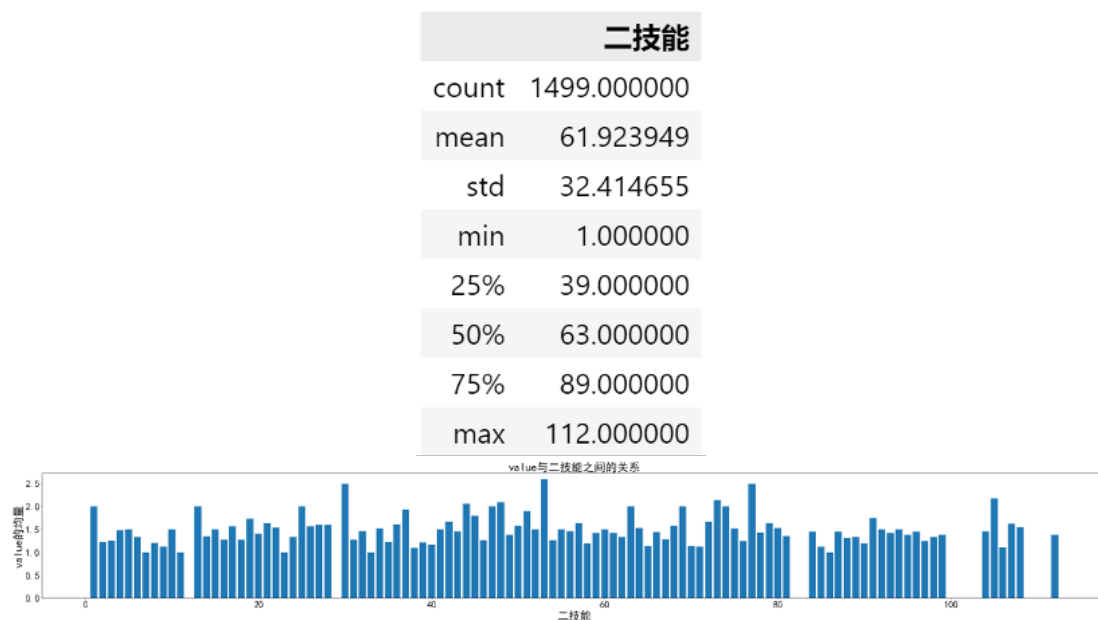


Figure 8: 技能二

2.3.4 护石等级与 value 的关系

护石的等级越高，有好护石的占比越高

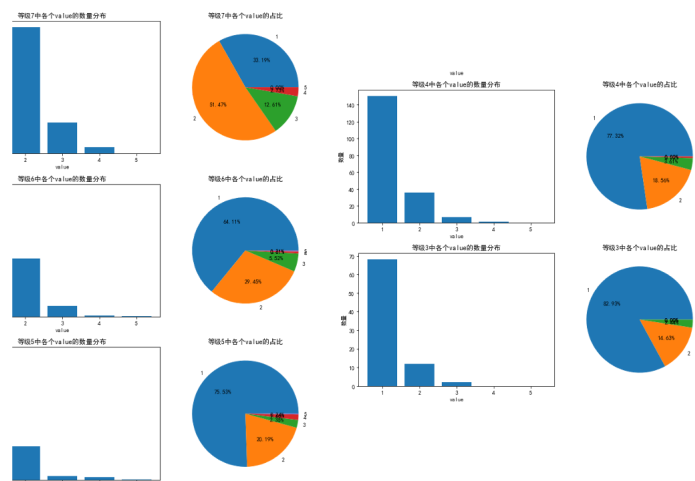


Figure 9: 护石等级与 value 的关系

2.4 特征提取

2.4.1 降维

因为一技能和它的等级，二技能和它的等级，都是有关联的，一技能是它等级的类别标签，二技能是它等级的类别标签，所以一技能和它的等级，二技能和它的等级之间最好使用线性判别分析（Linear Discriminant Analysis）进行降维，之后再将一技能和二技能之间使用 PCA 降维成一个整体

Listing 1: 降维

```
1 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
2 from sklearn.decomposition import PCA
3
4 lda = LDA(n_components=1)
5 pca = PCA(n_components=1)
6 skill1=stone[['一技能','Lv']]
7 re_1=lda.fit_transform(skill1,stone['一技能'])
8 skill2=stone[['二技能','Lv1']]
9 re_2=lda.fit_transform(skill2,stone['二技能'])
10 skill=np.concatenate((re_1,re_2),axis=1)
11 re_skill=pca.fit_transform(skill)
12 stone=pd.concat((stone,pd.DataFrame(re_skill,columns=['temp'])),axis = 1)
13 stone.drop(['一技能', 'Lv', '二技能', 'Lv1'], axis=1,inplace=True)
```

	等级	孔	value	temp
0	7	21	2	-1.848808
1	6	11	3	1.505373
2	5	11	1	-0.289074
3	6	11	1	-0.690824
4	7	1	4	2.214343
...	...	...	...	...
1494	5	11	1	-0.738088
1495	5	0	1	-1.068941
1496	7	31	2	-0.147280
1497	4	1	3	2.167078
1498	4	0	1	1.952777

1499 rows × 4 columns

Figure 10: 降维

2.4.2 降维

最后对于所需的数据进行归一化处理，以便提高精度

Listing 2: 降维

```

1 from sklearn.preprocessing import StandardScaler
2 kong=stone['孔'].values.reshape(-1,1)
3 temp=stone['temp'].values.reshape(-1,1)
4 rank=stone['等级'].values.reshape(-1,1)
5 scaler = StandardScaler(copy=False)
6 stone['孔']=scaler.fit_transform(kong)
7 stone['temp']=scaler.fit_transform(temp)
8 stone['等级']=scaler.fit_transform(rank)

```

	等级	孔	value	temp
0	1.147585	0.352653	2	-1.468677
1	0.319755	0.012831	3	1.195855
2	-0.508074	0.012831	1	-0.229638
3	0.319755	0.012831	1	-0.548784
4	1.147585	-0.326990	4	1.759054
...	...	...	...	...
1494	-0.508074	0.012831	1	-0.586331
1495	-0.508074	-0.360972	1	-0.849157
1496	1.147585	0.692474	2	-0.116998
1497	-1.335904	-0.326990	3	1.721508
1498	-1.335904	-0.360972	1	1.551269

1499 rows × 4 columns

Figure 11: 归一化

3 机器学习模型

3.1 半监督学习

半监督学习主要用于数据标签缺失的情况，只有一小部分标签数据和较大部分的未标签数据。而在我们所找到的数据集中只有游戏中所出现过的护石，其评分并没有被完全标记。而将数据全部标记，手动标签识别工作量大。为此我们采用了半监督的机器模型进行标签的分类识别。半监督识别有三大假设：

1. 平滑假设：相似的数据具有相同的 label
2. 聚类假设：处于同一个聚类下的数据具有相同 label
3. 流形假设：处于同一流形结构下的数据具有相同 label

在经过数据的基本分析后，数据集中的数据基本符合以上假设的先提条件。所以我们决定采用半监督模型来实现该项目。

3.2 标签传播算法

标签传播算法是半监督机器学习中的一个较为常见的算法，标签传播算法的核心思想为相似的数据应该具有相同的 label。标签传播算法包括两大步骤：

1. 构造相似矩阵
2. 标签传播

标签传播算法是一种图算法。我们为所有的数据构建一个图，图的节点就是一个数据点，包含 labeled 和 unlabeled 的数据。节点 i 和节点 j 的边表示他们的相似度。图的构建方法很多，若图是全连接的，节点 i 和节点 j 的边权重为：

$$W_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{\alpha^2}\right) \quad (1)$$

其中 α 为超参，为在调整模型时可以调整的参数。

在构建完图之后，通过节点之间的边传播 label。边的权重越大，表示两个节点越相似，那么 label 越容易传播过去。为此我们需要构建一个 $N \times N$ 的概率转移矩阵 P ：

$$P_{ij} = p(i \rightarrow j) = \frac{w_{ij}}{\sum_{k=1}^n w_{ik}} \quad (2)$$

P_{ij} 表示从节点 i 转移到节点 j 的概率。假设有 C 个类和 L 个 labeled 样本，定义一个 $L \times C$ 的 label 矩阵 Y_L ，第 i 行表示第 i 个样本的标签指示向量，即如果第 i 个样本的类别是 j ，那么该行的第 j 个元素为 1，其他为 0。同样给 U 个 unlabeled 样本一个 $U \times C$ 的 label 矩阵 Y_U 。合并后得到一个 $N \times C$ 的 soft label 矩阵 $F=[Y_L;Y_U]$ ，即保留样本 i 属于每个类别的概率，而非互斥性的，以概率 1 只属于一个类。最后确定样本 i 的类别时，取概率最大的那个类作为它的类别。

在完成传播后，要注意将 label 数据的类别复位以免被传播的标签影响。在不断的进行标签传播后，数据会趋向于收敛并得出答案。

3.3 sklearn 模块应用

在理解标签传播后，我们使用了 sklearn 中 semi_supervised 包的 LabelSpreading 类进行半监督学习的基础。

Listing 3: LabelSpreading

```
1 cls=LabelSpreading(max_iter=100,kernel='rbf',gamma=97,alpha=0.8811)
```

其中 max_iter 为标签传播的迭代次数，采用了高斯核，并且调整了其中的 γ 和 α 参数。

模型输入的数据使用了上文中特征提取部分处理后的数据，并将标签与数据进行分离。在模型的正确性判断上，我们使用了五百个数据进行“数据窥探”，虽然这会导致数据的过拟合，但这无法避免。

在使用模型生成数据后，我们也用同样的方式进行了检测，确保了模型的正确性，即传播的结果一致。

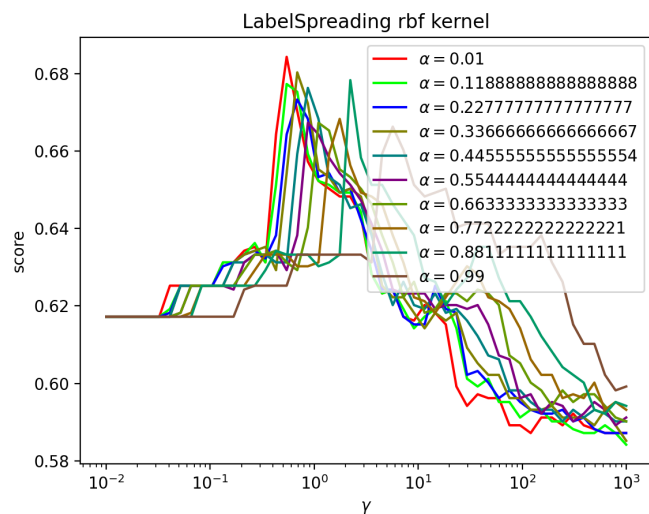


Figure 12: 参数调整

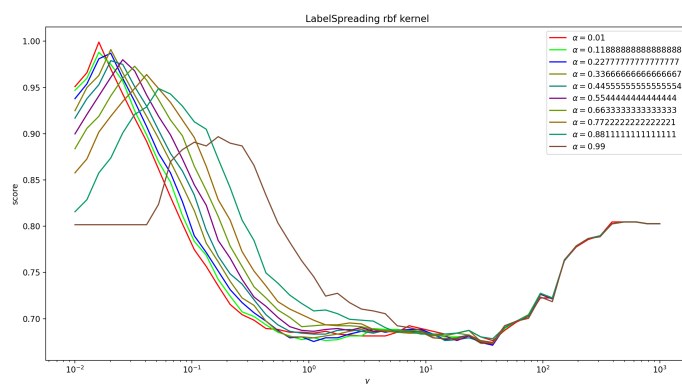


Figure 13: 结果验证

3.4 应用模型生成

在后期的模型使用过程中，我们发现了半监督模型直接作为应用模型的 bug，新数据进行降维后存在偏差，会导致结果的极大不同。为了排除这一影响，我们仅将半监督模型作为数据的标记部分，最后的应用模型则是使用了半监督模型所标记的数据以原数据使用各种机器学习模型进行测试并计算准确率，从中筛选出正确率较高的模型并作为最终的模型使用。

4 应用场景

4.1 GUI

实例：一个 tkinter 窗口、六个 label、六个 combobox 下拉框、一个按钮
布局：绝对布局，窗口固定

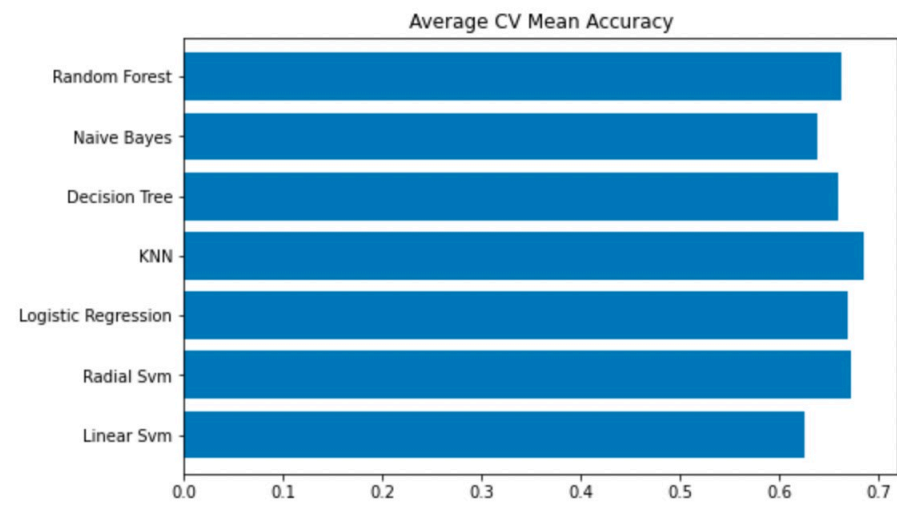


Figure 14: 生成模型

Listing 4: GUI

```

1 self.top = tk.Tk()
2 self.top.geometry("500x300")
3 self.top.resizable(0, 0)
4 self.top.title("Monster Hunter Stone")

```

事件：绑定在按钮上，当点击按钮时触发事件。

Listing 5: GUI

```

1 def init_button(self): # 初始化按钮并绑定action事件
2     self.button = tk.Button(self.top, text="评判", command=lambda: self.action()).
3         place(x=240, y=250)
4 def action(self):
5     TheStone = stone(self.value6.get(), self.value1.get(), self.value2.get(), self.
6         value3.get(), self.value4.get(), self.value5.get())
7     TheStone.judge()

```

护石类：

属性为需要输入的各种属性：技能 1、技能 1 等级、技能 2、技能 2 等级、孔位、稀有度。

4.2 护石判断

评价护石属性的方法：加载模型，将输入的数据加入到原数据集中进行一并处理（归一化、处理中文等），最后调用模型预测。

Listing 6: Judge

```

1 def judge(self):
2     estimator = joblib.load('cls.model') # 加载模型
3     skill = pd.read_excel("skill.xlsx")

```

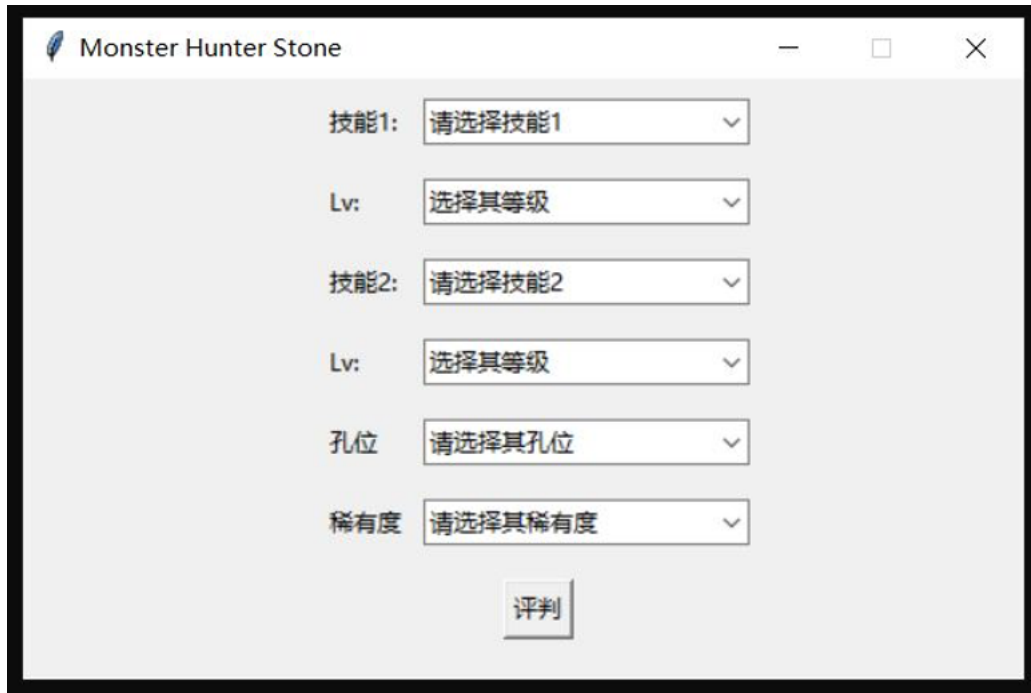


Figure 15: GUI

```

4     stone_old= pd.read_excel("半监督.xlsx")
5     stone_old=stone_old.iloc[:,6]
6     # print(stone_old.head())
7
8     stone = pd.DataFrame([[self.rarity, self.name1, self.lv1, self.name2, self.lv2,
9                             self.hole]], columns=['等级', '一技能', 'Lv', '二技能', 'Lv1', '孔'])
10    skillArray = skill["LookUpName"].values
11
12    stone = pd.concat((stone_old, stone), axis=0)
13
14    # 处理缺失值
15    stone.fillna({"二技能": "无", "Lv1": 0}, inplace=True)
16
17    #处理技能,将技能名称用数字替换
18    skillArray = skill["LookUpName"].values
19    stone['一技能'].replace(skillArray, list(range(1,113)), inplace=True)
20    stone['二技能'].replace(skillArray, list(range(1,113)), inplace=True)
21
22    lda = LDA(n_components=1)
23    pca = PCA(n_components=1)
24    skill1=stone[['一技能', 'Lv']]
25    re_1=lda.fit_transform(skill1, stone['一技能'])
26    skill2=stone[['二技能', 'Lv1']]
27    re_2=lda.fit_transform(skill2, stone['二技能'])

```

```

27     skill = np.concatenate((re_1, re_2), axis=1)
28     re_skill = pca.fit_transform(skill)
29     stone=stone.reset_index()
30     temp= pd.DataFrame(re_skill, columns=['temp'])
31     temp=temp.reset_index()
32     df = pd.concat((stone,temp), axis=1)
33     print(df)
34     df.drop(['一技能','Lv','二技能','Lv1'],axis = 1,inplace=True)
35     stone = df
36     #数据归一化
37     kong=stone['孔'].values.reshape(-1,1)
38     temp=stone['temp'].values.reshape(-1,1)
39     rank=stone['等级'].values.reshape(-1,1)
40
41     scaler = StandardScaler(copy=False)
42     stone['孔']=scaler.fit_transform(kong)
43     stone['temp']=scaler.fit_transform(temp)
44     stone['等级']=scaler.fit_transform(rank)
45
46     data=stone.iloc[[-1]]
47     del data['index']
48     print(data)
49     tk.messagebox.showinfo('结果', estimator.predict(data))

```

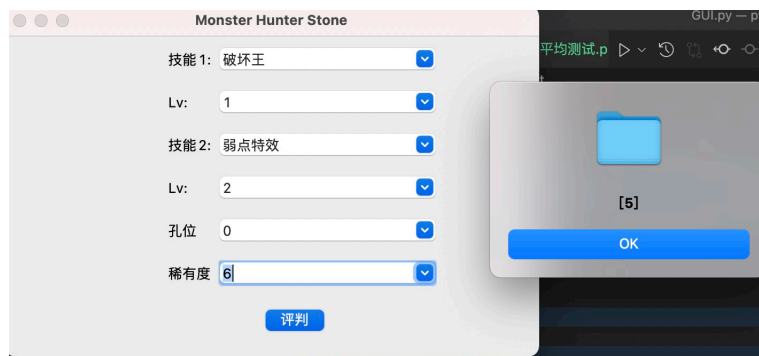


Figure 16: 运行结果

5 特色

1. 充分应用课程知识，串联各章节重点进行数据分析处理
2. 利用 LDA, PCA 降维与归一化进行数据预处理
3. 使用半监督模型进行机器学习，学习使用 LabelSpread 模型
4. 数据可视化与界面可视化完善
5. 创新性强，核心内容原创
6. 具有一定的实际应用价值
7. 分工明确，使用 git 同步项目，团队合作性好 (<https://gitee.com/tchar7/python-analysis>)

6 任务分工

1. 唐晨宇 (组长): 机器学习模型, 报告主笔
2. 沈豪: 数据收集处理
3. 沈宇涵: 特征分析处理
4. 陈高廷: GUI

7 分析总结

经过本次的课程设计，我们能够结合课堂知识，以此为基础去探寻一些新的应用。我们结合了数据处理、数据分析与机器学习，能够初步对一些我们日常中所遇到的问题进行分析，并且能够有所应用，初步达到了课程目的。

但除此之外，我们也还有许多不足，例如很多原理的不清晰，对问题的应变能力还有待提升。在我们的程序中也有很多值得改进的地方。

在整个流程中，我们充分认识到了数据在机器学习中起到决定性的作用，在我们的数据中，标记的数据较少，会导致最后的结果偏差较大，过拟合的现象可能比较严重。在最开始的标记数据过程中，应当选择一些比较有特征的数据进行数据标记而不是仅进行前段数据的标记，也因此出现了“数据窥视”的问题。在后期使用模型时因为对于数据将有的理解不够到位，出现了输出结果的错误，只能使用原数据进行预测，暂时没能很好的解决这个问题，希望在日后的学习中能够得到答案。

改进方案

1. 对于数据集进行更好的处理
2. 处理数据降维后的问题
3. 优化 GUI 的约束条件，约束用户的输入
4. 对用户的输入数据也要进行预处理
5. GUI 与数据可视化结合

References

- [1] 标签传播算法 <https://blog.csdn.net/zouxy09/article/details/49105265>